

Fast16: Pre-Stuxnet Sabotage Tool Was Built to Subvert Nuclear Weapons Simulations

✓ security.com/threat-intelligence/fast16-nuclear-sabotage

Key findings

- Fast16's hook engine is selectively interested in high-explosive simulations inside LS-DYNA and AUTODYN.
- All evidence suggests that attackers were specifically targeting simulations of nuclear detonations.
- The malware checks for the density of the material being simulated and only acts when that value passes 30 g/cm³, the threshold uranium can only reach under the shock compression of an implosion device.
- Up to ten distinct software builds carry tailored hooks, suggesting a sustained operation that tracked target organisations' software updates over years.
- The tampering only activates during full-scale transient blast and detonation runs.
- Fast16 propagates within a target network using share enumeration and impersonation but is built not to leave that network.

Overview

In April 2026, our peers in SentinelOne published the first public analysis of fast16, a previously undiscovered sabotage framework whose oldest components appear to date from around 2005, approximately two years before Stuxnet first became active. The framework consists of a service binary that embeds an early Lua 5.0 virtual machine, a boot-start filesystem driver that intercepts and patches executable code as it is read from disk, and a rule-driven hook engine that rewrites very specific instruction sequences inside a single, narrowly defined target application.

Symantec's Threat Hunter Team has reviewed fast16's hook engine and can confirm LS-DYNA and AUTODYN as targeted applications. Both are software applications used to simulate real-world problems such as vehicle crashworthiness, material modelling, and explosive simulation. In addition to this, we have discovered that fast16 was built to tamper with simulations of high-explosive detonations, almost certainly in support of strategic sabotage against nuclear weapons research.

Confirming the targeted application

Once the driver is installed, it creates a kernel file system filter to monitor all accessed files. It first waits until EXPLORER.EXE is launched and then targets any files with the extension .EXE that are compiled with the Intel compiler by matching the string 'Intel' in the PE header. Whenever such a file is read into memory, the hook engine performs on-the-fly patching if it contains matching opcode sequences. The hook engine inside fast16 has a table of 101 byte-pattern rules. Each rule fires when a specific instruction sequence is read from disk and either captures an absolute address or places a hook to malicious code in an injected .xdata section.

Below as an example are rules 46 and 47, which capture and overwrite a very specific x87 floating-point sequence:

```
fstp dword [ebp+0]      ; store result to caller's REAL*4 out-arg fld  dword
[abs]                  ; reload from array fld  dword [imm32]      ; static global
REAL*4 constant – captured fmul dword [imm32]      ; multiply by second global
REAL*4 constant ; followed by far-call to injected handler
```

The patterns the rules match against do not match against every Intel-Fortran-compiled, single-precision, explicit-dynamics solver of that era, but are found in versions of LS-DYNA and AUTODYN. These patterns are specific to different versions of the software and some could belong to other simulation programs as well.

Fast16's end goal

The most interesting question is what fast16 is doing inside LS-DYNA or AUTODYN once it has installed itself. The hooks fast16 places inside of the simulation program consist of three attack strategies. We will refer to these as Mechanism A, B, and C. All appear to target simulations of high-pressure shock behavior.

Mechanism A

The first and 16th times Mechanism A's hook point is reached, fast16 simply returns control. Otherwise, if the simulation input value is between 30 and 65, fast16 scales the output values down to 10% of their normal value and continues to keep it at 10% of their normal values thereafter. One can imagine a hypothetical graph such as this:

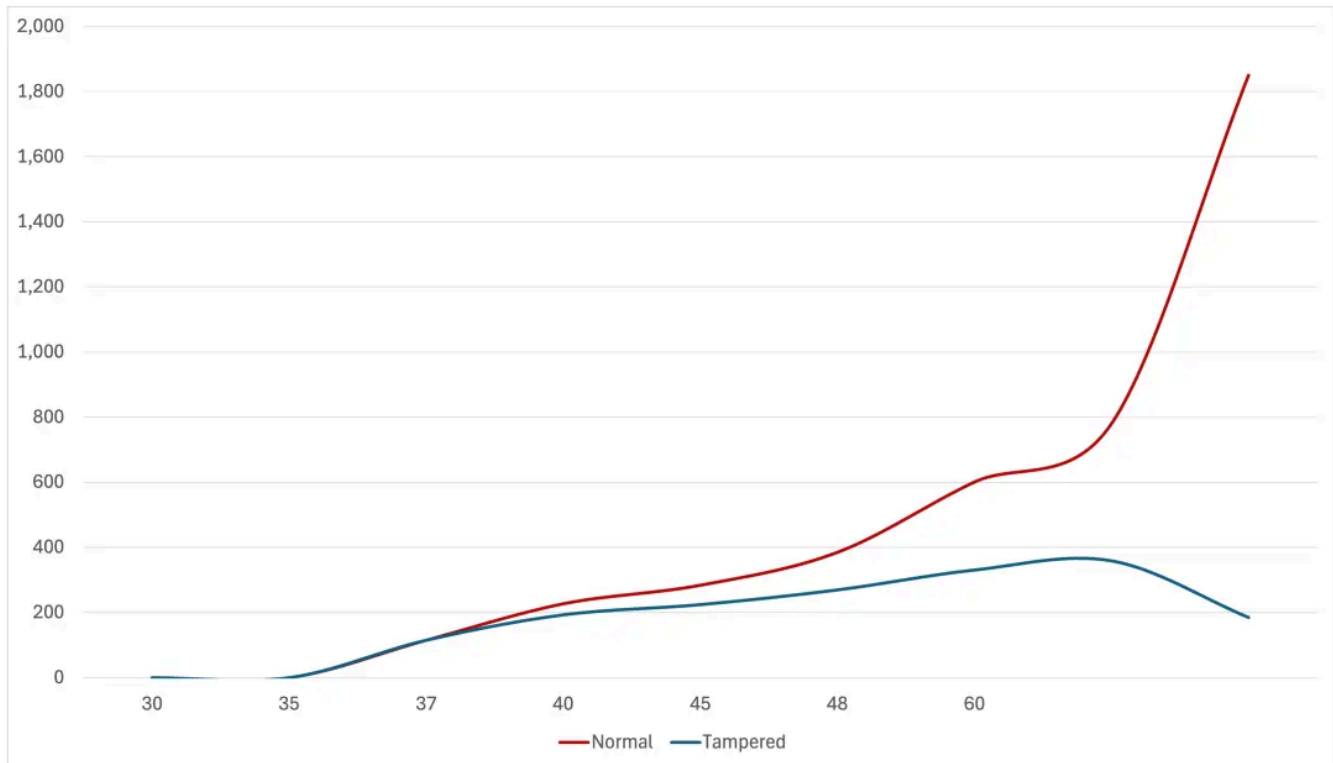


Figure 1. Hypothetical graph of normal vs tampered values.

Mechanism B

The hook points for mechanism B, designed for LS-DYNA, first check if the Equation of State (EOS) selection is 2 (Jones-Wilkins-Lee), 3 (Sack Tuesday), or 7 (Ignition and Growth of Reaction in High Explosives). An EOS is a mathematical model that determines how a material's pressure changes when its volume or density is compressed or expanded. These particular models are for modelling high explosives. If the EOS selection is met, the code will only begin its tampering routine if certain attributes of the simulation reach five times its initial value. Then, the Cauchy stress tensor output values (sig_xx, sig_yy, sig_zz) of any model run after are modified down to 1% of their true values if the density of the material reaches 30g/cm³. The code does not immediately reduce these output values to 1%. Instead, the code 'naturally' scales the reduction to 1% by calculating the slope required to reach 1% by the time the density is 60g/cm³. Note the simulation does not need to reach 60g/cm³. Based on the values utilized, the material being targeted is uranium and the Cauchy stress tensor values will represent the thermodynamic pressure of the material, which determine compressibility of the material. In testing, this resulted in increased compression of the material than actuality when modeling compression to 33g/cm³.

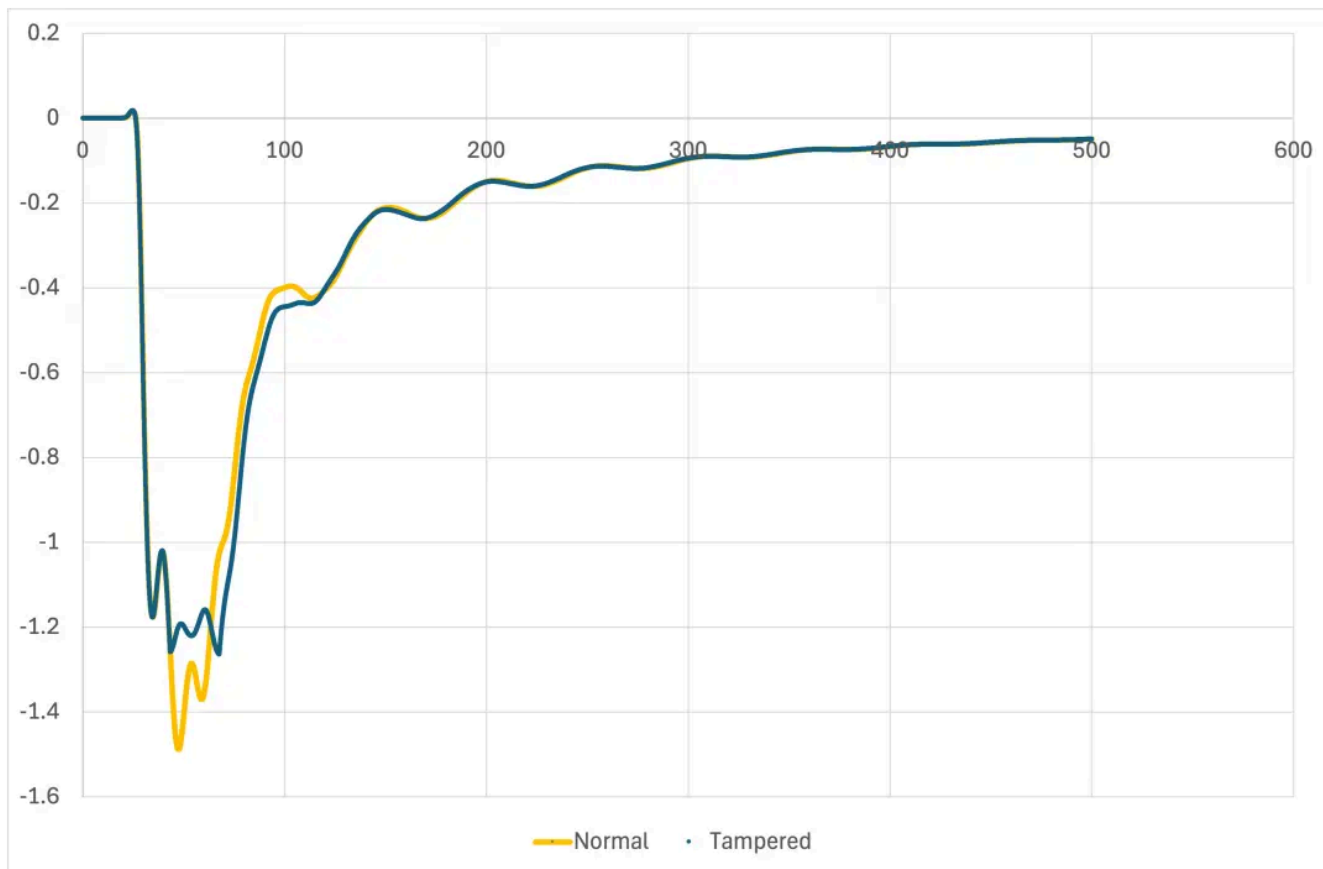


Figure 2. Tampered test results.

Mechanism C

The hook points for mechanism C are designed for AUTODYN and first check for the value of 3, 5, 11. In AUTODYN, these are EOS Ideal Gas (3), JWL (5), and Lee-Tarver (11).

The mechanism for C will also not act unless a certain attribute of the simulation reaches five times its initial value (the threshold value) and the string “\$Loading co” is found in memory, which is found in AUTODYN. Depending on the version of the simulation software, Mechanism C will scale an output value (such as Pressure) at different rates. The decrease in output values start when the current uranium density in the successful simulation reaches a compression of 30g/cm³ and scales the output values to variable end densities as detailed in the below table. The simulation does not need to reach the end density, which is only used to calculate the slope at which to perform the reduction.

Start Density	End Density	Tamp Down (% of true value)
30g/cm ³	60g/cm ³	42%

Start Density	End Density	Tamp Down (% of true value)
30g/cm ³	40g/cm ³	10%
30g/cm ³	47g/cm ³	10%
30g/cm ³	48g/cm ³	8%

One may imagine at different stages of design, the targets were using the current version of the simulation software at that time, to which the attackers devised different tampering methods relative to the simulations being conducted at that time. In fact, the 101 hook rules can be separated further into 9-10 hook groups, each for a different build of LS-DYNA or AUTODYN.

Targeting of nuclear detonation simulations

Normally, neutrons leak out of the uranium without hitting anything, but when the uranium is compressed, these neutrons are more likely to strike another uranium nucleus. This causes the nucleus to split (fission) and also leak more neutrons, which then strike further uranium nuclei, causing an explosive chain reaction. For nuclear weapons, high explosives are placed around the uranium core. The high explosives cause a pressure wave that compresses the uranium, initiating a nuclear explosion.

When simulating the performance of uranium within a nuclear weapon, one simulates the pressure and the material effects to determine if the uranium will become compressed enough to reach supercriticality, when neutrons are introduced that cause a nuclear explosion. The specific EOS models, material models, and scaling factors chosen align with simulation of performance of uranium within a nuclear weapon. Mechanism B and C initially requires a high explosive simulation and then tampers with the pressure simulation of uranium. All the tampering mechanisms effectively reduce the output values such as the Cauchy stress tensor to disrupt the simulation.

How the simulation users reacted to the simulation modifications remains unclear. While output values and graphs may seem reasonable to a layman, inconsistencies in results may still be apparent to an expert. With only single output values being modified, they can lead to unexpected feedback and inconsistencies in the model and inconsistencies in dependent values.

Two pieces of anecdotal evidence may support the effect was simply to impair the operation of the software, sometimes noticeably. If hook rule groups were added sequentially as needed, we see a hook group added for a previous version of the software after a newer

version. One may imagine, the simulation user reverted to an older version when faced with the anomaly, before that version was also targeted. Secondly, the hook groups represent up to 10 different versions of simulation software, meaning the simulation user updated versions semi-frequently.

Nevertheless, whether the results showed supercriticality when it was not actually reached or failure when supercriticality was actually reached or simple statistical anomalies, fast16 would likely have delayed and disrupted the successful construction of a nuclear weapon.

Fast16 installer

Fast16 was designed to be installed and spread within a target network, but not outside that network. The initial executable, svcmgmt.exe, exposes five modes via command-line arguments:

- No arguments runs it as a Windows service
- p - handles remote installation and execution
- i - installs and runs Lua code,
- r - runs the Lua code
- when args ≥ 2 – for use when hijacking a legitimate Windows program (IFEO Persistence)

The Lua code provides fast16's main execution behavior through 13 libraries covering host operations, remote service control, registry manipulation, and an embedded resource container.

Before installing, the script checks the NtfsMetaDataMutex to ensure a single instance and scans for 18 endpoint security registry keys, refusing to propagate if any are present. Under the install flags, fast16 copies itself to %windir%\system32\svcmgmt.exe, timestamps the file by cloning creation dates and ACL permissions from services.exe, and registers itself as the SvcMgmt service. It then drops the fast16.sys kernel driver into the system drivers folder, matches its timestamps to beep.sys, and configures the registry to load it as a SCSI-class filter driver on the next boot.

For persistence, fast16 abuses Image File Execution Options by writing its own path into the Debugger value under HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<target>, which causes Windows to launch fast16 instead of the chosen application. On execution, the malware deletes that registry key, launches the original application, re-adds the key to maintain persistence, and then re-runs itself with the 'r' command-line argument for normal execution. The user sees a working application while the hijack is silently restored.

To spread, fast16 extracts svcmgmt.dll and registers it as a Multiple Provider Router (MPR) network provider notifyee, causing Windows to load the DLL into any process that calls WNetAddConnection and report new share connections back via the named pipe \\.\pipe\p577. In parallel, fast16 enumerates all domains, servers, and shares to discover further remote hosts. Each candidate hostname is resolved to an IP and checked against the local network ranges (10.x.x.x, 172.16.x.x, 192.168.x.x) and the same-subnet rule. For machines that qualify, fast16 impersonates the locally logged-on user's credentials, copies itself to \\<remote>\admin\$\system32\svcmgmt.exe, and creates a remote SvcMgmt service to start execution on the new host. After a configured sleep, it repeats the cycle to find additional machines to infect.

Significance

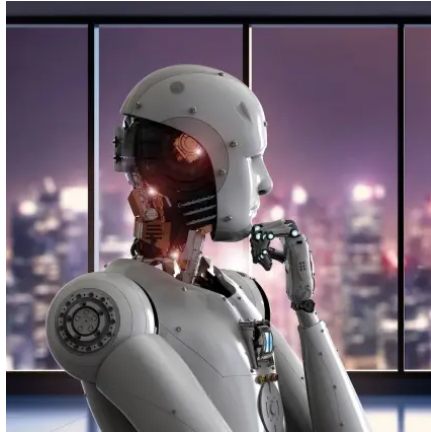
Fast16 targets, with rule-level precision, the key models for detonation of nuclear weapons. That degree of domain knowledge, such as understanding which EOS forms matter, which calling conventions are produced by which compilers, and which classes of simulation will or will not trip the gate, is unusual in any era and was very unusual in 2005. The framework belongs to the same conceptual lineage as Stuxnet, in which malware was tailored not just to a vendor's product but to a specific physical process being simulated or controlled by that product.

Defenses

We do not know if a modern-day version of fast16 exists. Organizations concerned about similar threats capable of sabotage should iterate regularly across their endpoints to inventory loaded drivers, flagging any that are unsigned or unfamiliar. Application control should be deployed and tightly tuned to block unapproved executables and DLLs from running, denying attackers the chance to drop and side-load custom tooling in the first place. Symantec Endpoint Security and Carbon Black EDR should both be in use across the estate, with Symantec Endpoint Security's Adaptive Protection feature enabled to harden the system, for example by denying the use of dual-use tool not normally used in the environment, without breaking legitimate administration.

We'd like to thank the Institute for Science and International Security for their expertise and invaluable insights that informed this piece.

About the Author



Threat Hunter Team

Symantec and Carbon Black

The Threat Hunter Team is a group of security experts within Broadcom whose mission is to investigate targeted attacks, drive enhanced protection in Symantec and Carbon Black products, and offer analysis that helps customers respond to attacks.